

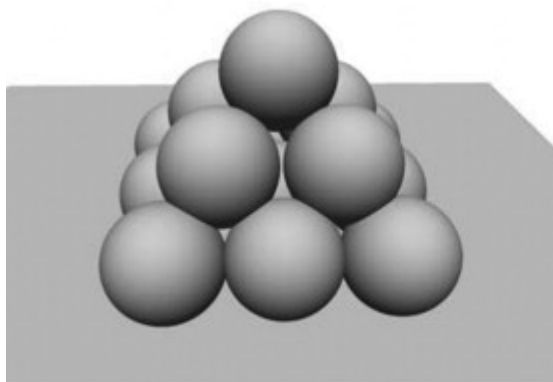
SSAO Assignment Guideline

1st session: Two-step renderer

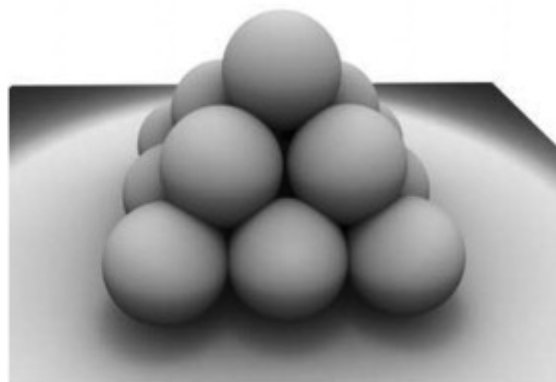
In this session, we will create a two-step renderer that allows us to compute and visualize all the data we need to estimate Ambient Occlusion (AO) in Screen-Space (SS).

Introduction to Screen Space Ambient Occlusion

Ambient occlusion is an approximation of the amount by which a point on a surface is occluded by the surrounding geometry, which affects the accessibility of that point by the incoming light. In effect, ambient occlusion techniques allow the simulation of proximity shadows - the soft shadows that you see in the corners of rooms and the narrow spaces between objects. Ambient occlusion is often subtle, but will dramatically improve the visual realism of a computer-generated scene.

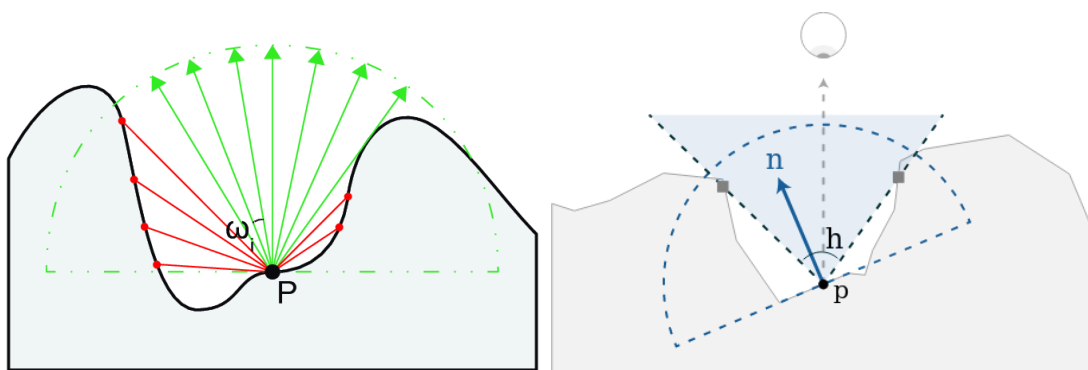


Without ambient occlusion



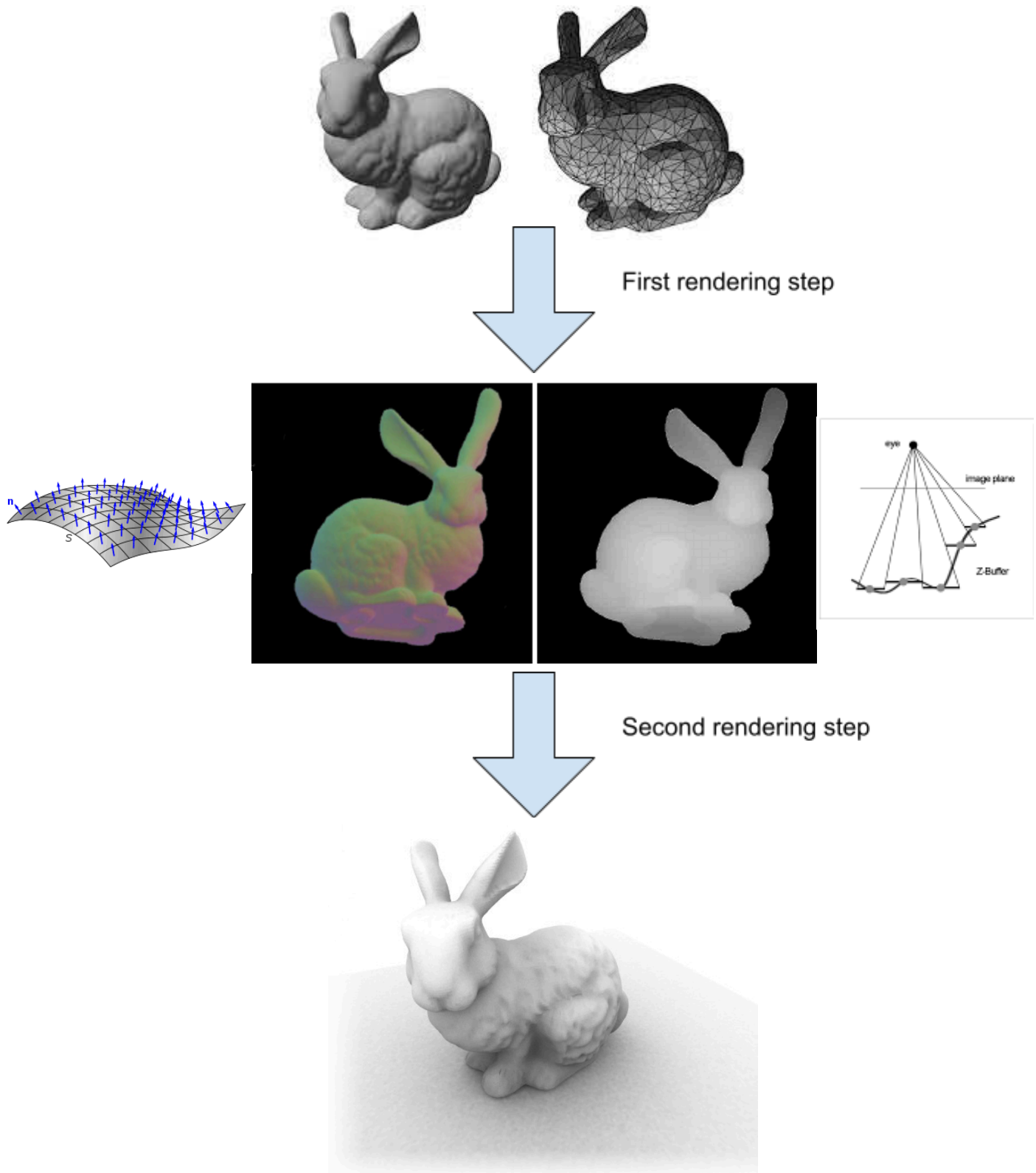
With ambient occlusion

The basic idea is to compute an *occlusion factor* for each point on a surface and incorporate this into the lighting model, usually by modulating the ambient term such that more occlusion = less light, less occlusion = more light. The *occlusion factor* indicates the percentage of the hemisphere described by the normal of the surface that is occluded by other parts of the geometry.



Computing the occlusion factor can be expensive; offline renderers typically do it by casting many rays in a normal-oriented hemisphere to sample the occluding geometry around a point. In general, this isn't practical for real-time rendering.

Instead, the idea of SSAO is to compute a simplification of the scene by computing the normal and the position of each fragment of the viewport. Then, it approximates the AO of each fragment by sampling the surrounding fragments.



Rendering the first step

During the first rendering step, we will render the geometry of the scene. The idea is that for each fragment, we will compute its albedo, normal, and depth. We will store this information in textures using FrameBuffers and color information. To achieve that, you will need to follow the following steps:

At the beginning of the execution:

- generate the textures where you will store the information
 - Related functions: `glGenTextures`, `glBindTexture`, `glTexImage2D`, `glTexParameter`
- generate the FrameBuffers and attach the textures to them
 - Related functions: `glGenFramebuffers`, `glBindFramebuffer`, `glFramebufferTexture2D`,
- generate the RenderBuffers and attach the textures to them (optional)
 - Related functions: `glGenRenderbuffers`, `glBindRenderbuffer`, `glRenderbufferStorage`, `glFramebufferRenderbuffer`
- define the array of color buffers where the fragment shader will draw into
 - Related functions: `glDrawBuffers`

During the rendering loop:

- bind the corresponding frame buffer.
 - Related functions: `glBindFramebuffer`
- clear the buffers (at least color and depth), send the uniforms, and draw the geometry.
 - Related functions: `glClear`, `glGetUniformLocation`, `glUniform*`, `glBindVertexArray`, `glDrawElements`

Rendering the second step

During the second rendering step, we will use the information stored in the textures to generate the final image. It is not necessary to render again the scene, as all the needed information is in the textures. However, we must ensure that all the needed fragments are rendered. To achieve that, we will render a quad that completely fills the viewport. After that, we will send the texture information to the shaders and the shader will compute the corresponding final image. Remember that the GUI may allow the user to choose between displaying the albedo, the normal, or the depth. You will need to consider the following steps:

At the beginning of the execution:

- Generate the structures (VAO, VBO) to render the quad.
 - Related functions: `glGenVertexArrays`, `glGenBuffers`, `glBindBuffer`, `glBufferData`, `glEnableVertexAttribArray`, `glVertexAttribPointer`

During the rendering loop:

- clear the buffers (at least color and depth), send the uniforms, and draw the geometry.
 - Related functions: `glClear`, `glGetUniformLocation`, `glUniform*`, `glActiveTexture`, `glBindTexture`, `glBindVertexArray`, `glDrawElements`